UNIT 2

LEXICAL UNITS :

In PL/SQL (Procedural Language/Structured Query Language), lexical units are the basic elements from which programs are constructed. These units include identifiers, keywords, literals, and special characters. Here's a breakdown of each:

1. **Identifiers**: Identifiers are names given to variables, constants, functions, procedures, cursors, and other database objects. They must adhere to certain rules:
    - They must begin with a letter.
    - They can be followed by letters, digits, dollar signs ($), or underscores
    - They cannot exceed 30 characters in length.

2.Keywords: Keywords are reserved words in PL/SQL that have predefined meanings and cannot be used as identifiers. Some common keywords include `BEGIN`, `DECLARE`, `END`, `IF`, `LOOP`, `WHILE`, `FOR`, `SELECT`, `UPDATE`, `INSERT`, `DELETE`, etc.

1. **Literals**: Literals are constant values that are used directly in PL/SQL code. There are several types of literals:
    - **String literals**: Enclosed in single quotes (`'`). Example: `'Hello, World!'`
    - **Numeric literals**: Represented directly as numbers. Example: `42`, `3.14`
    - **Boolean literals**: Represented by `TRUE` or `FALSE`.
    - **Date literals**: Represented by a date value enclosed in single quotes with the format `'YYYY-MM-DD'`. Example: `'2024-03-07'`
    - **Null literals**: Represented by `NULL`, indicating the absence of a value.

3.Special Characters: These include symbols used for arithmetic operations, comparison, assignment, and logical operations. Some common special characters in PL/SQL include `+`, `-`, `*`, `/` for arithmetic operations, `=`, `!=`, `<`, `>`, `<=`, `>=` for comparison, `:=` for assignment, `AND`, `OR`, `NOT` for logical operations, etc.

2. **Conditional Statements**:
    - **IF-THEN-ELSE**: Executes a block of code if a condition is true, otherwise executes an alternative block of code.
    - **CASE**: Allows for conditional branching based on the value of an expression.
3. **Looping Statements**:
    - **LOOP**: Executes a block of code repeatedly until a condition is met.
    - **WHILE**: Executes a block of code repeatedly as long as a condition is true.
    - **FOR**: Executes a block of code a specified number of times.
4. **Branching Statements**:
    - **GOTO**: Transfers control to a labeled statement in the same block or subprogram.
    - **EXIT**: Exits the current loop or block.
    - **CONTINUE**: Skips the remaining statements in the current iteration of a loop and proceeds to the next iteration.

CURSORS

In PL/SQL, cursor structures are used to manage the result sets returned by queries. Cursors provide a mechanism for iterative processing of query results row by row.

1. **Implicit Cursors**:
    - Implicit cursors are automatically created by PL/SQL whenever a SQL statement is executed, and they are used for queries that return a single row or execute single-row DML (Data Manipulation Language) statements like INSERT, UPDATE, DELETE, or SELECT INTO.
    - Implicit cursors do not need to be explicitly declared or opened. PL/SQL manages them behind the scenes.
    - However, implicit cursors lack the flexibility and control provided by explicit cursors, especially when dealing with multi-row result sets.

```
DECLARE

v_employee_name employees.first_name%TYPE;

BEGIN

SELECT first_name INTO v_employee_name FROM employees WHERE employee_id =
100;

DBMS_OUTPUT.PUT_LINE('Employee name: ' || v_employee_name);
END;
/
```

2. **Explicit Cursors**:
    - Explicit cursors are defined and managed explicitly by the programmer. They provide more control over the result set processing compared to implicit cursors, especially for queries that return multiple rows.
    - Explicit cursors are declared, opened, fetched, and closed explicitly by the programmer.
    - They are suitable for situations where you need to process multiple rows returned by a query.

```
DECLARE
   CURSOR emp_cursor IS
      SELECT employee_id, first_name, last_name FROM employees;
   v_employee_id employees.employee_id%TYPE;
   v_first_name employees.first_name%TYPE;
   v_last_name employees.last_name%TYPE;
BEGIN
   OPEN emp_cursor; LOOP
      FETCH emp_cursor INTO v_employee_id, v_first_name, v_last_name;
      EXIT WHEN emp_cursor%NOTFOUND;
      DBMS_OUTPUT.PUT_LINE('Employee ID: ' || v_employee_id || ', Name: ' ||
v_first_name || ' ' || v_last_name);
   END LOOP;
   CLOSE emp_cursor;
END;
```

BULK STATEMENTS

In PL/SQL, bulk processing refers to the handling of multiple rows of data at once, rather than processing them individually. This approach can significantly improve performance by reducing the number of context switches between the PL/SQL engine and the SQL engine, thus minimizing overhead.

Bulk processing is typically achieved using bulk SQL statements and collections. Here are the main components involved:

1. **Bulk SQL Statements**:
    - Bulk SQL statements allow you to perform operations on multiple rows of data with a single SQL statement, rather than executing the statement for each row individually.
    - Examples of bulk SQL statements include:
        - `BULK COLLECT INTO`: Fetches multiple rows of data from a query into a collection.
        - `FORALL`: Performs DML operations (INSERT, UPDATE, DELETE) on collections of data in a single operation.
2. **Collections**:
    - Collections are PL/SQL data types that can hold multiple values.
    - Common types of collections used for bulk processing include:
        - **Associative Arrays (Index-By Tables)**: Collections indexed by integer or string keys.
        - **Nested Tables**: Unordered collections with a single column of data.
        - **VARRAYs (Variable-Size Arrays)**: Arrays with a fixed upper limit.

COLLECTIONS

Collections in PL/SQL are composite data types that allow you to store multiple values of the same or different data types in a single variable. They provide a way to manipulate sets of data more efficiently than using individual scalar variables. There are three main types of collections in PL/SQL:

1. **Associative Arrays (Index-By Tables)**:

   - Also known as index-by tables or simply arrays, associative arrays are collections that are indexed by a unique key. This key can be either a numeric or string value.

   - Associative arrays are particularly useful when you need to store and manipulate data in a tabular format.

   - They can be sparse (with gaps between index values) or dense (with consecutive index values).

```sql
DECLARE
    TYPE salary_table IS TABLE OF NUMBER INDEX BY VARCHAR2(100);
    emp_salaries salary_table;
BEGIN
    emp_salaries('John') := 50000;
    emp_salaries('Alice') := 60000;
    emp_salaries('Bob') := 55000;
    DBMS_OUTPUT.PUT_LINE('John''s Salary: ' || emp_salaries('John'));
END;
```

2. **Nested Tables**:

   - Nested tables are collections that can hold an arbitrary number of elements, similar to arrays in other programming languages.

   - They are ordered collections with a single column of data.

- Nested tables can be sparse or dense, and they can be stored in the database as database objects.

```sql
DECLARE
    TYPE num_list IS TABLE OF NUMBER;
    my_numbers num_list := num_list(1, 2, 3, 4, 5);
BEGIN
    FOR i IN 1..my_numbers.COUNT LOOP
        DBMS_OUTPUT.PUT_LINE('Number: ' || my_numbers(i));
    END LOOP;
END;
```

3. **VARRAYs (Variable-Size Arrays)**:

- VARRAYs are arrays with a fixed upper limit on the number of elements they can contain.

- Unlike nested tables, varrays have a maximum size specified at declaration.

- They are useful when you know the maximum number of elements you need to store in advance.

Example of declaring and using a VARRAY:

```
DECLARE
    TYPE phone_numbers IS VARRAY(3) OF VARCHAR2(15);
    contacts phone_numbers := phone_numbers('123-456-7890', '456-789-1230', '789-123-4560');
BEGIN
    FOR i IN 1..contacts.COUNT LOOP
```

```
    DBMS_OUTPUT.PUT_LINE('Phone Number: ' || contacts(i));

  END LOOP;

END;
```

Varray and Table Collections IN PL/SQL

In PL/SQL, VARRAYs (Variable-Size Arrays) and nested tables are two types of collections used to store multiple values in a single variable. Both types have their own characteristics and use cases.

1. **VARRAY (Variable-Size Array)**:
   - VARRAY is a collection type with a fixed maximum size specified at declaration.
   - It behaves similar to arrays in other programming languages.
   - Elements of a VARRAY are accessed by their index, starting from 1.
   - VARRAYs are particularly useful when you know the maximum number of elements you need to store and that limit doesn't change.
   - You cannot extend or shrink a VARRAY beyond its declared size.
   - VARRAYs are stored inline in the database table structure.

Example of a VARRAY declaration and usage:

```
CREATE OR REPLACE TYPE phone_numbers_type AS VARRAY(3) OF VARCHAR2(15);
/
DECLARE
   contacts phone_numbers_type := phone_numbers_type('123-456-7890', '456-789-1230', '789-123-4560');
BEGIN
   FOR i IN 1..contacts.COUNT LOOP
      DBMS_OUTPUT.PUT_LINE('Phone Number: ' || contacts(i));
   END LOOP;
END;
/
```

2. **Nested Table**:
   - Nested tables are dynamic arrays without a fixed size limit.
   - They can hold an arbitrary number of elements.
   - Nested tables are stored out of line in the database, similar to regular database tables.
   - Nested tables can be sparse (with gaps between index values) or dense (with consecutive index values).
   - Nested tables can be used to represent one-to-many relationships or sets of related data.

Example of a nested table declaration and usage:

```sql
CREATE OR REPLACE TYPE num_list_type AS TABLE OF NUMBER;
/
DECLARE
   my_numbers num_list_type := num_list_type(1, 2, 3, 4, 5);
BEGIN
   FOR i IN 1..my_numbers.COUNT LOOP
```

```
        DBMS_OUTPUT.PUT_LINE('Number: ' || my_numbers(i));
    END LOOP;
END;
/
```

In summary, VARRAYs are suitable when you know the maximum number of elements and the size won't change, while nested tables are more flexible and can grow or shrink dynamically. Both types of collections provide powerful ways to handle multiple values in PL/SQL. The choice between them depends on your specific requirements and use case.

ASSOCIATIVE ARRAYS

Associative arrays, also known as index-by tables, are a type of collection in PL/SQL that allow you to store data in a flexible manner using key-value pairs. They are somewhat similar to dictionaries or hash tables in other programming languages. Associative arrays are particularly useful when you need to store and manipulate data in a tabular format where the index values are not necessarily sequential or numeric.

Here's an overview of associative arrays in PL/SQL:

1. **Declaration**:

- You declare an associative array by specifying the data type of the elements and the data type of the index.
- The index can be either a numeric type or a string type.

Example of declaring an associative array:

```sql
DECLARE
    TYPE salary_table IS TABLE OF NUMBER INDEX BY VARCHAR2(100);
    emp_salaries salary_table;
BEGIN
    -- Associative array declared
END;
/
```

3. **Initialization**:

- After declaring an associative array, you can initialize it with data using assignment statements.

Example of initializing an associative array:

```sql
emp_salaries('John') := 50000;
emp_salaries('Alice') := 60000;
emp_salaries('Bob') := 55000;
```

4. **Accessing Elements**:

- You access elements of an associative array using the index value associated with each element.

Example of accessing elements of an associative array:

```sql
DBMS_OUTPUT.PUT_LINE('John"s Salary: ' || emp_salaries('John'));
```

5. **Iteration**:

- You can iterate over the elements of an associative array using loops.

Example of iterating over an associative array:

```sql
FOR emp_name IN emp_salaries.FIRST..emp_salaries.LAST LOOP
   DBMS_OUTPUT.PUT_LINE(emp_name || '"s Salary: ' || emp_salaries(emp_name));
END LOOP;
```

5. **Methods**:

- Associative arrays support several methods and attributes for manipulation, such as `COUNT`, `EXISTS`, `DELETE`, etc.

Example of using the `COUNT` attribute:

```sql
FOR i IN 1..emp_salaries.COUNT LOOP
   DBMS_OUTPUT.PUT_LINE('Employee ' || i || ' Salary: ' || emp_salaries(i));
END LOOP;
```

Associative arrays provide a flexible and efficient way to store and manipulate data in PL/SQL. They are especially useful when you need to associate arbitrary keys with values and when the order of elements is not important.

ORACLE COLLECTION API

In PL/SQL, Oracle provides a built-in Collection API, which is a set of functions and procedures for working with collections efficiently. These collection methods are available for both associative arrays and nested tables. They provide convenient ways to manipulate and retrieve data stored in collections. Here's an overview of some of the commonly used methods in the Collection API:

COUNT

EXIST

LIMIT

PRIOR AND NEXT

FIRST AND LAST